EXPERIMENT - 06

```
import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import os
IMAGE_PATH = "/workspaces/computer-vision-Experiments./EXP_06_OG_IMAGE.jpg"
OUTPUT_DIR = "output_exp6"
os.makedirs(OUTPUT_DIR, exist_ok=True)
# STEP 1: LOAD ORIGINAL IMAGE
image = cv2.imread(IMAGE_PATH)
if image is None:
    raise FileNotFoundError(
        "Image not found. Make sure image.jpg is in the project folder."
    )
cv2.imwrite(
    os.path.join(OUTPUT_DIR, "01_original_image.png"),
    image
)
```



STEP 2: GRAYSCALE CONVERSION AND GAUSSIAN BLUR

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)
cv2.imwrite(
    os.path.join(OUTPUT_DIR, "02_grayscale_blurred.png"),
    blurred
)
```



```
# STEP 3: GLOBAL THRESHOLDING
# WORD OUTPUT: FIGURE 3 - GLOBAL THRESHOLDING RESULT
_, global_threshold = cv2.threshold(
    blurred,
    127,
    255,
    cv2.THRESH_BINARY
)
cv2.imwrite(
    os.path.join(OUTPUT_DIR, "03_global_threshold.png"),
    global_threshold
)
```

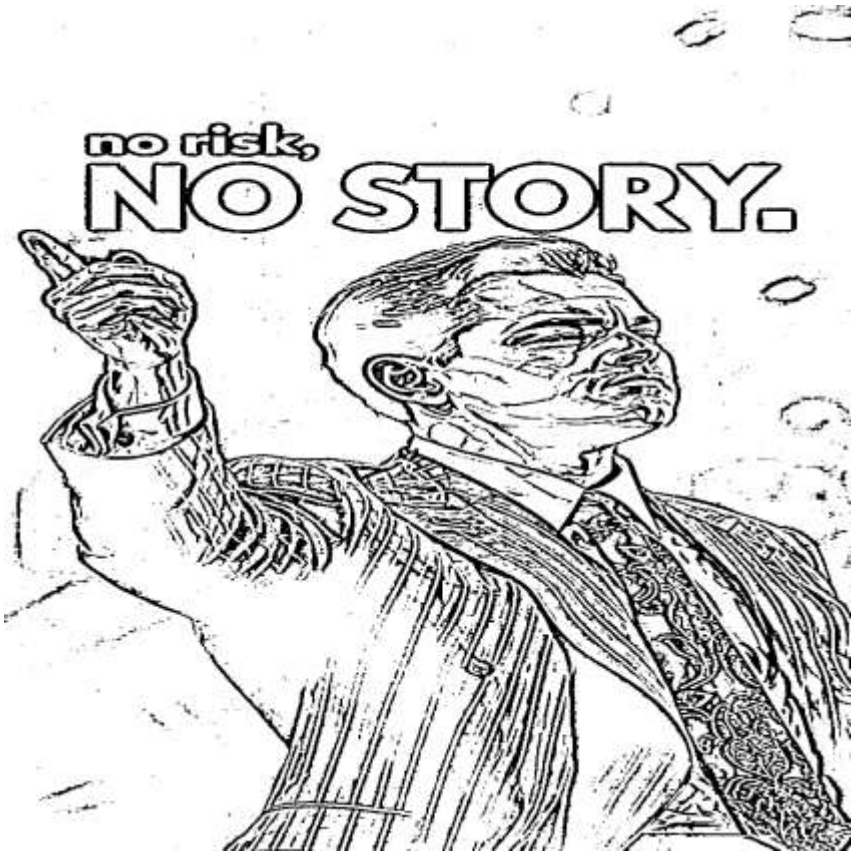


```
# STEP 4: OTSU'S THRESHOLDING
otsu_value, otsu_threshold = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
cv2.imwrite(
    os.path.join(OUTPUT_DIR, "04_otsu_threshold.png"),
    otsu_threshold
)
```




```
# STEP 5: ADAPTIVE THRESHOLDING
adaptive_threshold = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
```

```
    cv2.THRESH_BINARY,
    11,
    2
)
cv2.imwrite(
    os.path.join(OUTPUT_DIR, "05_adaptive_threshold.png"),
    adaptive_threshold
)
```




```
# STEP 6: WATERSHED SEGMENTATION
_, binary = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)
kernel = np.ones(
    (3, 3),
```

```
    np.uint8
)
opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)
sure_background = cv2.dilate(
    opening,
    kernel,
    iterations=3
)
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)
_, sure_foreground = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)
sure_foreground = np.uint8(
    sure_foreground
)
unknown = cv2.subtract(
    sure_background,
    sure_foreground
)
num_labels, markers = cv2.connectedComponents(
    sure_foreground)
```

```
markers = markers + 1
markers[unknown == 255] = 0
watershed_image = image.copy()
markers = cv2.watershed(
    watershed_image,
    markers
)
watershed_result = image.copy()
watershed_result[markers == -1] = [
    0,
    0,
    255
]
cv2.imwrite(
    os.path.join(
        OUTPUT_DIR,
        "06_watershed_segmentation.png"
    ),
    watershed_result
)
```



```
# STEP 7: K-MEANS CLUSTERING
image_rgb = cv2.cvtColor(
    image,
    cv2.COLOR_BGR2RGB
)
pixels = image_rgb.reshape(
    (-1, 3)
)
sample_size = min(
    50000,
    len(pixels)
)
np.random.seed(42)
indices = np.random.choice(
    len(pixels),
    sample_size,
    replace=False
)
sample_pixels = pixels[
    indices
]
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(
    sample_pixels
)
labels = kmeans.predict(
    pixels
)
```

```
centers = np.uint8(
    kmeans.cluster_centers_
)
segmented_pixels = centers[
    labels
]
kmeans_result = segmented_pixels.reshape(
    image_rgb.shape
)
kmeans_bgr = cv2.cvtColor(
    kmeans_result,
    cv2.COLOR_RGB2BGR
)
cv2.imwrite(
    os.path.join(
        OUTPUT_DIR,
        "07_kmeans_segmentation.png"
    ),
    kmeans_bgr
)
```



```
# STEP 8: COMPARATIVE VISUALIZATION
plt.figure(
    figsize=(12, 8)
)
plt.subplot(2, 3, 1)
plt.imshow(
    cv2.cvtColor(
        image,
        cv2.COLOR_BGR2RGB
    )
)
plt.title("Original Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(
    global_threshold,
    cmap="gray"
)
plt.title("Global Thresholding")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.imshow(
    otsu_threshold,
    cmap="gray"
)
plt.title("Otsu Thresholding")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.imshow(
    adaptive_threshold,
    cmap="gray"
)
plt.title("Adaptive Thresholding")
```

```
plt.axis("off")
plt.subplot(2, 3, 5)
plt.imshow(
    cv2.cvtColor(
        watershed_result,
        cv2.COLOR_BGR2RGB ))
plt.title("Watershed")
plt.axis("off")
plt.subplot(2, 3, 6)
plt.imshow(
    kmeans_result)
plt.title("K-Means")
plt.axis("off")
plt.tight_layout()
plt.savefig(
    os.path.join(
        OUTPUT_DIR,
        "08_segmentation_comparison.png" ),
    dpi=200,
    bbox_inches="tight"
)plt.close()
```



```
# STEP 9: SAVE NUMERICAL K-MEANS DATA
np.save(
    os.path.join(
        OUTPUT_DIR,
        "kmeans_labels.npy"
    ),
    labels
)
np.save(
    os.path.join(
        OUTPUT_DIR,
        "kmeans_centers.npy"
    ),
    centers
)
# STEP 10: DISPLAY RESULTS IN TERMINAL
print("=" * 60)
print("EXPERIMENT NO. 6")
print("IMAGE SEGMENTATION TECHNIQUES")
print("=" * 60)
print("\nImage Shape:", image.shape)
print(
    "Global Threshold Value:",
    127
)
print(
    "Otsu Optimal Threshold Value:",
    otsu_value
)
print(
    "K-Means Number of Clusters:",
    3
)
```

```
print(
    "Watershed Regions:",
    num_labels - 1
)
print("\nSegmentation techniques implemented:")
print("1. Global Thresholding")
print("2. Otsu's Thresholding")
print("3. Adaptive Thresholding")
print("4. Watershed Segmentation")
print("5. K-Means Clustering")
print("\nOutput files saved in:", OUTPUT_DIR)
print("\nExperiment completed successfully.")
```

EXPERIMENT NO. 6 — QUESTIONS & ANSWERS

Q1. What is Image Segmentation? Why is it considered a fundamental step in computer vision?

Answer:
Image segmentation is the process of dividing an image into meaningful regions or objects based on properties such as intensity, color, texture, or boundaries. It helps separate the **foreground/object from the background.**

It is a fundamental step in computer vision because it makes further tasks such as **object detection, image recognition, medical analysis, and scene understanding** easier and more accurate.

Example: Segmenting a medical image to separate a tumor region from healthy tissue.

Q2. Differentiate between Image Segmentation and Image Classification.

Answer:

Image Segmentation	**Image Classification**
Divides an image into meaningful regions or objects.	Assigns a label or class to an entire image.
Works at pixel/region level.	Usually works at image level.
Provides information about the location and shape of objects.	Provides information about what the image contains.
Example: Separating a car from the road.	Example: Classifying an image as "car".

Example: Segmentation identifies **where** the car is, while classification identifies **what** the image contains.

Q3. Explain the working principle of Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Answer:

1. **Global Thresholding:** Uses a fixed threshold value for the entire image. Pixels above or below the threshold are assigned to different regions.
   Example: Threshold value = 127.
2. **Otsu's Thresholding:** Automatically selects an optimal threshold value by analyzing the image histogram and separating the image into foreground and background.
3. **Adaptive Thresholding:** Calculates different threshold values for different local regions of the image. It is useful when illumination is uneven.

Example: Adaptive thresholding can be used to segment text from a document having shadows or uneven lighting.

Q4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Answer:
The **Watershed Algorithm** is a region-based image segmentation technique that treats the image like a topographic surface and separates different regions using boundaries.

It is particularly useful for separating **touching or overlapping objects** because it creates boundaries between connected regions.

Example: Separating two touching cells in a microscopic image.

Q5. How is K-Means Clustering applied to image segmentation?

Answer:
K-Means is an unsupervised clustering technique used to divide image pixels into **K groups or clusters** based on their similarity.

The steps are:

1. Select the number of clusters **K**.
2. Assign pixels to the nearest cluster center.
3. Calculate new cluster centers.
4. Repeat until the centers become stable.
5. Replace each pixel with its cluster center to obtain the segmented image.

Example: Using K-Means with **K = 3** to divide an image into three color-based regions.

Q6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Answer:

Threshold-Based Segmentation	**Clustering-Based Segmentation**
Separates pixels using threshold values.	Groups similar pixels into clusters.
Usually simpler and faster.	Generally more computationally expensive.
Works well when foreground and background have clear intensity differences.	Works well for images with multiple colors or regions.
Examples: Global, Otsu, Adaptive Thresholding.	Example: K-Means Clustering.

Example: Otsu's method separates foreground/background, while K-Means can divide an image into multiple color regions.

Q7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Answer:
Some common challenges are:

- **Uneven illumination** can make foreground and background difficult to distinguish.
- **Complex backgrounds** may have colors or textures similar to the object.
- **Noise** can create unwanted regions.
- Objects may have **weak or unclear boundaries.**
- Overlapping objects can be difficult to separate.

Example: A person wearing clothes similar in color to the background can be difficult to segment accurately.

Q8. Mention five real-world applications where image segmentation plays a critical role.

Answer:
Five important applications are:

1. **Medical imaging** – detecting tumors and organs.
2. **Autonomous vehicles** – identifying roads, vehicles, and pedestrians.
3. **Satellite image analysis** – identifying land, water, and buildings.
4. **Industrial inspection** – detecting defects in products.
5. **Agriculture** – identifying crops, weeds, and diseased areas.

Example: In medical imaging, segmentation can isolate a tumor from surrounding healthy tissue.

Q9. How does image segmentation improve the performance of object detection and image recognition systems?

Answer:
Image segmentation separates the **region of interest from irrelevant background information.** This allows computer vision systems to focus on important objects and their boundaries.

As a result, segmentation can improve **object localization, recognition accuracy, and feature extraction.**

Example: In autonomous driving, segmenting the road and vehicles helps the system understand the driving environment more effectively.

Q10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Answer:

Traditional Segmentation	**Deep Learning Segmentation**
Uses manually designed techniques.	Learns features automatically from data.
Examples: Thresholding, Watershed, K-Means.	Examples: U-Net, Mask R-CNN.
Usually requires less computational power.	Generally requires more computational resources.
Works well for simple and controlled images.	Performs better on complex real-world images.
May require manual parameter selection.	Learns useful features during training.

Example: Thresholding can segment a simple black-and-white object, while **U-Net** can accurately segment structures in complex medical images.
